

Cross Validation

90

What is Cross Validation (CV)?

- Splits the data into **K parts** (folds) (like 5 or 10 pieces).
- Trains the model **K times**, each time using a different piece as the test set.
- **Averages** the performance across all folds.

91

Build Using Cross Validation

- import pandas as pd
- from sklearn.model_selection import cross_val_score, StratifiedKFold
- from sklearn.tree import DecisionTreeClassifier

92

Build Using Cross Validation

- model = DecisionTreeClassifier(max_depth=5, random_state=42)

93

Perform Cross Validation

- `cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`
- `cv_scores = cross_val_score(model, X, y, cv=cv, scoring='accuracy')`
- `print("Cross-Validation Accuracy Scores:", cv_scores)`
- `print("Average CV Accuracy:", cv_scores.mean())`

94

Product Recommendation System

- You are given two datasets
- **products.csv**
 - productId
 - category
 - price
- **purchases.csv**
 - userId
 - productId
 - rating
- Display the **Top 10 recommended products**

95

Product Recommendation System

- import pandas as pd
- from sklearn.model_selection import train_test_split
- from sklearn.naive_bayes import GaussianNB
- from sklearn.metrics import accuracy_score, precision_score, recall_score

96

Product Recommendation System

```
products = pd.read_csv('products.csv')
purchases = pd.read_csv('purchases.csv')
df = pd.merge(purchases, products, on='productId')
df['user_like'] = df['rating'].apply(lambda x: 1 if x >= 4 else 0)
df['category_code'] = df['category'].astype('category').cat.codes
X = df[['category_code', 'price']]
y = df['user_like']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
model = GaussianNB()
model.fit(X_train, y_train)
```

97

Product Recommendation System

```
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)

df['like_probability'] = model.predict_proba(X)[:, 1]
top_products = df[['productId', 'category', 'like_probability']] \
    .drop_duplicates() \
    .sort_values('like_probability', ascending=False)
print(top_products.head(10))
```